Arrays:

s++ s e--

0 1 2 3 4

1	2	3	4	5

e

reverse: 5, 4, 3, 2, 1

reverse(arr)

4 → 0

arr[i]

void swap(int a, int b)
tem = a
a = b
b = tem

swap(arr[s], arr[e])

e

5 4 3 2 1

e = 0

s

arr = [2]

s > e

s = 0

while(s <= e)

1, 2, 3, 4, 5

Rotate Left

rotateleft(arr, k)

5, 6

k=1 2, 3, 4, 5, 1

k=2 3, 4, 5, 1, 2

k=3 4, 5, 1, 2, 3

k=4 5, 1, 2, 3, 4

k=5, 1, 2, 3, 4, 5

k=6

Normalization:

n

k > arr.size()

6 > 5

$6 \% 5$

$(k \% n)$

[k = k % n]

= 6 % 5 = 1

7 % 5 = 2

k=1

Input → 1, 2, 3, 4, 5

0 1 2 3 4

k=1

O/p = 5, 1, 2, 3, 4

0 1 2 3 4

0 – 4

(4+1)

Out[(i+K)%n]

i=0 1 = 1

i=1 2 = 2

i=2 3 = 3

i=3 4 = 4

i=4 5 % 5 = 0

o[1] = 1

o[2] = 2

o[3] = 3

o[4] = 4

o[0] = 5

In = 4

In = 0

In = 1

In = 2

In = 3

Out → 0

Out → 1

Out → 2

Out → 3

Out → 4

arr[i]

a[0] = 1

a[1] = 2

a[2] = 3

a[3] = 4

a[4] = 5

in → [1, 2, 3, 4, 5]

0 1 2 3 4

K = K % n

out = | | | | | |

0 1 2 3 4

i → 0–4

out[i] = in[(i+k) % n]

Left Rotation ⇒ out[(i+k)%n] = in[i]

out[1] = 1 = in[0] = 1

out[2] = 2 = in[1] = 2

out[3] = 3 = in[2] = 3

out[4] = 4 = in[3] = 4

out[0] = 5 = in[4] = 5

1, 2, 3, 4, 5 →

5, 1, 2, 3, 4

Left

k=1 1, 2, 3, 4, 5 ←

2, 3, 4, 5, 1

k=2 3, 4, 5, 1, 2

4, 5, 1, 2, 3

60 →

⇒ Leet Code

⇒ Coding Ninjas

Collection Framework

STL

→ Stacks

→ queues

BinarySearch [100]

Matrix:

r c

3 × 4 Matrix

n × n

m × n Matrix?

3 R

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 6 & 7 \end{bmatrix}$$

4 C

Dynamically

m n

int arr[3][4] rows

static